

Лямбды в C++

Замыкания, захват, generic lambdas, type erasure

Пучкин Станислав (СПбГУ)

C++ Семинар

9 апреля 2026 г.

```
1 // Без захвата: анонимная структура + приведение к func*
2 auto adder = [](int x, int y) { return x + y; };
3
4 // Компилятор генерирует примерно:
5 struct Closure {
6     static int func(int x, int y) { return x + y; }
7     operator decltype(&func)() const { return func; }
8 };
9 // Поэтому работает неявное приведение:
10 int (*pf)(int, int) = adder; // ok
11
12 // Тип уникален - переопределить нельзя:
13 auto f = []{};
14 f = []{}; // ошибка: несовместимые типы
15
16 // Unary+ форсирует приведение к func* (positive hack):
17 auto g = +[]{};
18 g = []{}; // ok - g теперь указатель на функцию
```

Что и как захватывается

```
1  int g = 1;
2  void foo() {
3      int x = 10; static int s = 2;
4
5      auto lam = [=] { return x + s + g; };
6      auto lam2 = [&] { return x + s + g; };
7      auto lam3 = [&x, copy_s=s, copy_g=g] {
8          return x + copy_s + copy_g;
9      };
10     x = 99;
11     std::cout << lam() << std::endl; // 13
12     std::cout << lam2() << std::endl; // 102
13     std::cout << lam3() << std::endl; // 102
14     s = 10;
15     std::cout << lam() << std::endl; // 21
16     std::cout << lam2() << std::endl; // 110
17     std::cout << lam3() << std::endl; // 102
18 }
19 // Правило: захватывается только локальный
20 // нестатический контекст. static и глобальные
21 // доступны напрямую и изменения в них всегда видны.
```

```
1 #include <iostream>
2
3 auto foo(int x = 10) {
4     return [&]{return x;};
5 }
6
7 int main() {
8     auto f = foo();
9     std::cout << f(); // 32766
10    return 0;
11 }
```

Взятие по ссылке в многопоточном коде

```
1 void byref() {
2     std::vector<std::thread> threads;
3     int i;
4     for (i = 0; i < 10; i++) {
5         threads.emplace_back([&i] {
6             std::cout << i << '␣'; });
7     }
8     for (auto &th : threads) { th.join(); }
9     // 2 6 7 7 8 99 10 10 10
10    std::cout << '\n';
11 }
12
13 void bycopy() {
14     std::vector<std::thread> threads;
15     for (int i = 0; i < 10; i++) {
16         threads.emplace_back([i] {
17             std::cout << i << '␣'; });
18     }
19     for (auto &th : threads) { th.join(); }
20     // 0 1 2 3 4 5 6 7 9 8
21     std::cout << '\n';
22 }
```

mutable и захват с переименованием

```
1 // [a] → operator() const: нельзя менять копию
2 // [a]() mutable → без const: можно
3 auto counter = [a = 0]() mutable { return ++a; };
4 // counter() → 1, counter() → 2, ...
5
6 // Захват по const-ссылке -: нужно переименование (C++14)
7 int x = 5;
8 auto lmd = [&rx = std::as_const(x), vx = x] {
9     return vx + rx; // rx: const int&
10 };
11 // lmd() → 10, но rx нельзя изменить
12
13 // Move-захват (C++14 init-capture)
14 auto ptr = std::make_unique<int>(42);
15 auto f = [p = std::move(ptr)]() { return *p; };
16 // ptr == nullptr после move
17 // f() == 42
```

Generic lambdas (C++14 / C++20)

```
1 // C++14: auto параметр- → шаблонный operator()
2 auto adder = [](auto x, auto y) { return x + y; };
3 adder(1, 2);          // int → 3
4 adder(1.5, 2.5);      // double → 4.0
5
6 // C++20: явные шаблонные параметры + concepts
7 template <typename T, typename U>
8 concept Addable = requires(T t, U u) { t + u; };
9
10 auto typed = []<typename T, typename U>(T x, U y)
11     requires Addable<T, U> { return x + y; };
12
13 typed(1, 2);          // ok
14 // typed("a", 1);    // ошибка: нет Addable<const char*,int>
15
16 // Доступ к типу T по имени внутри тела:
17 auto front = []<typename T>(std::vector<T>& v) -> T& {
18     return v.front();
19 };
```

“Перегрузка” лямбд (overload trick)

```
1 // Наследование от нескольких лямбд (C++17)
2 template <typename... F>
3 struct overload : F... { using F::operator()...; };
4 template<class... Ts>
5 overload(Ts...) -> overload<Ts...>; // deduction guide
6
7 auto f = overload {
8     [](int i)      { std::cout << "int_" << i; },
9     [](double d) { std::cout << "double_" << d; },
10 };
11 f(42); f(3.14); // int 42 / double 3.14
12
13 // std::variant + overload ⚡ паттернматчинг-
14 std::variant<int, double, std::string> v = 3.14;
15 std::visit(overload {
16     [](int i)      { std::cout << i; },
17     [](double d)    { std::cout << d; },
18     [](std::string& s){ std::cout << s; }
19 }, v); // выведет: 3.14
```


std::function: стирание типов и рекурсия

```
1 // auto: нулевой overhead, конкретный тип лямбды
2 auto fast = [x = 5](int a) { return x + a; };
3
4 // std::function: type erasure, heap indirection
5 std::function<int(int)> flex = [x = 5](int a) {
6     return x + a;
7 }; // внутри - void* + виртуальный вызов
8
9 // Рекурсивная лямбда требует стирания типа:
10 // auto fact = [&fact](int n){...}; // ошибка: fact ещё неизвестен
11 std::function<int(int)> fact = [&fact](int n) {
12     return n <= 1 ? 1 : n * fact(n - 1);
13 };
14 fact(5); // 120
15 // Внимание: fact захвачена по ссылке - не перемещать!
16
17 // Deducing this
18 auto fact = [](this auto&& self, int n) -> int {
19     return n <= 1 ? 1 : n * self(n - 1);
20 };
21 fact(5); // 120
```